Supervised Learning: Support Vector Machines & K Nearest Neighbor

Instructor: Dr. Chao Huang

Department of Computer Science
Montclair State University

Content partially contributed by Dr. Aparna Varde and Tilani Gunawardena

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- We talked about two supervised learning algorithms
  - Decision trees: recursively split data into branches based on feature values, leading to a decision outcome at each leaf
  - Naive Bayes: classifies data by calculating the probability of each class given the features (assuming all features are independent)

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- We talked about two supervised learning algorithms
  - Decision trees: recursively split data into branches based on feature values, leading to a decision outcome at each leaf
  - Naive Bayes: classifies data by calculating the probability of each class given the features (assuming all features are independent)

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- We talked about two supervised learning algorithms
  - Decision trees: recursively split data into branches based on feature values, leading to a decision outcome at each leaf
  - Naive Bayes: classifies data by calculating the probability of each class given the features (assuming all features are independent)

Recall

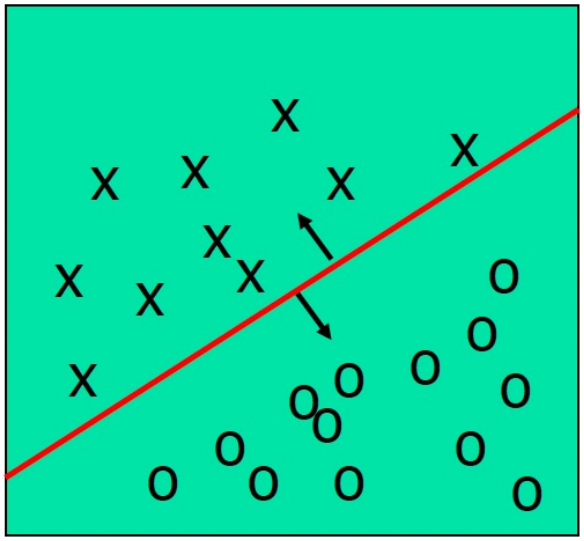
- What is supervised learning?
  - Labels of training data are provided in advance
- We talked about two supervised learning algorithms
  - Decision trees: recursively split data into branches based on feature values, leading to a decision outcome at each leaf
  - Naive Bayes: classifies data by calculating the probability of each class given the features (assuming all features are independent)

Outline

- Support Vector Machines (SVM)
- k-Nearest Neighbors (KNN)

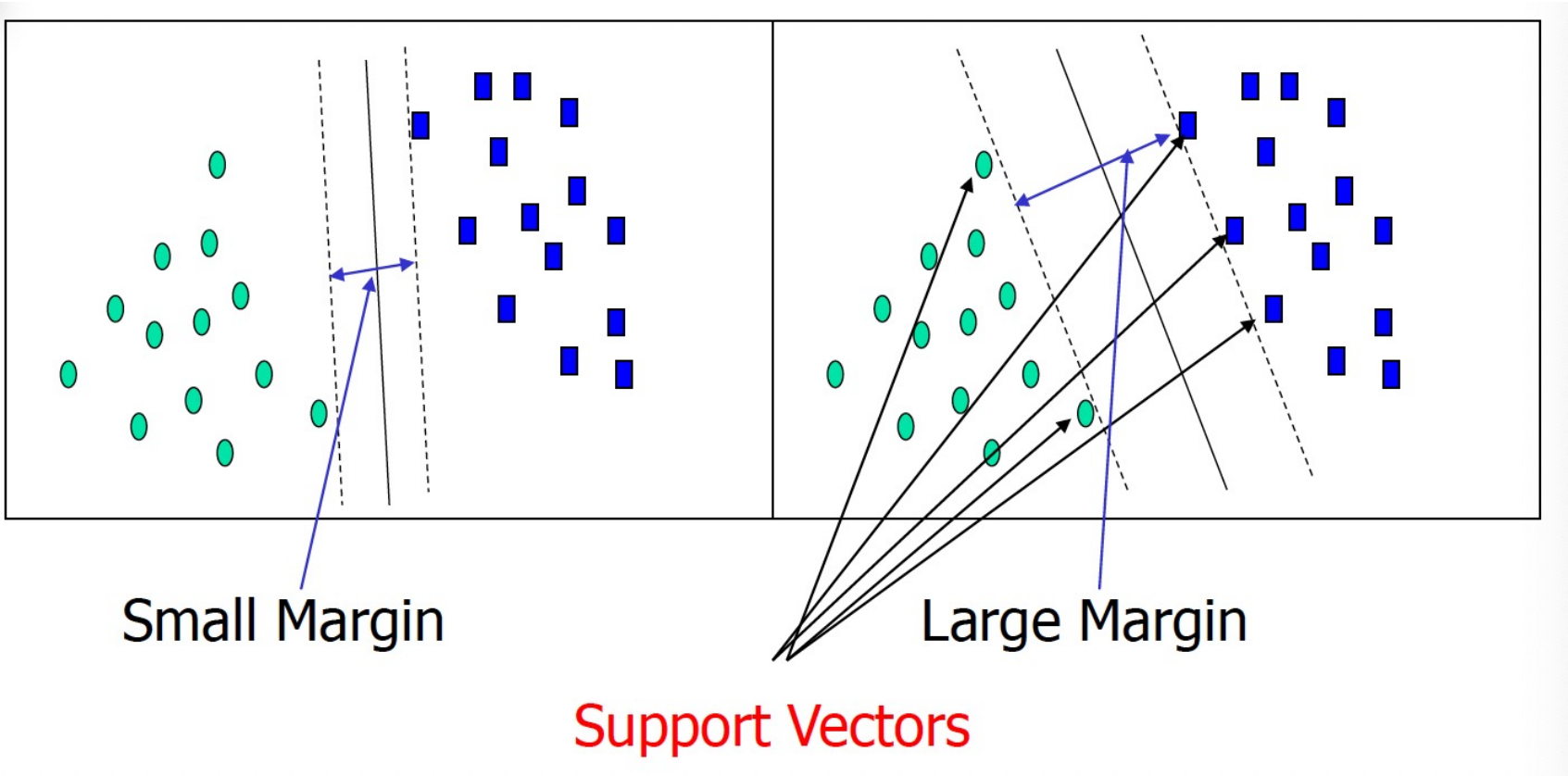
Support Vector Machines (SVM)

- A relatively new classification method for both linear and nonlinear data
- It uses a nonlinear mapping to transform the original training data into a higher dimension
- With the new dimension, it searches for the linear optimal separating **hyperplane** (i.e., "decision boundary")
- With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can always be separated by a hyperplane
- SVM finds this hyperplane using **support vectors** ("essential" training tuples) and **margins** (defined by the support vectors)

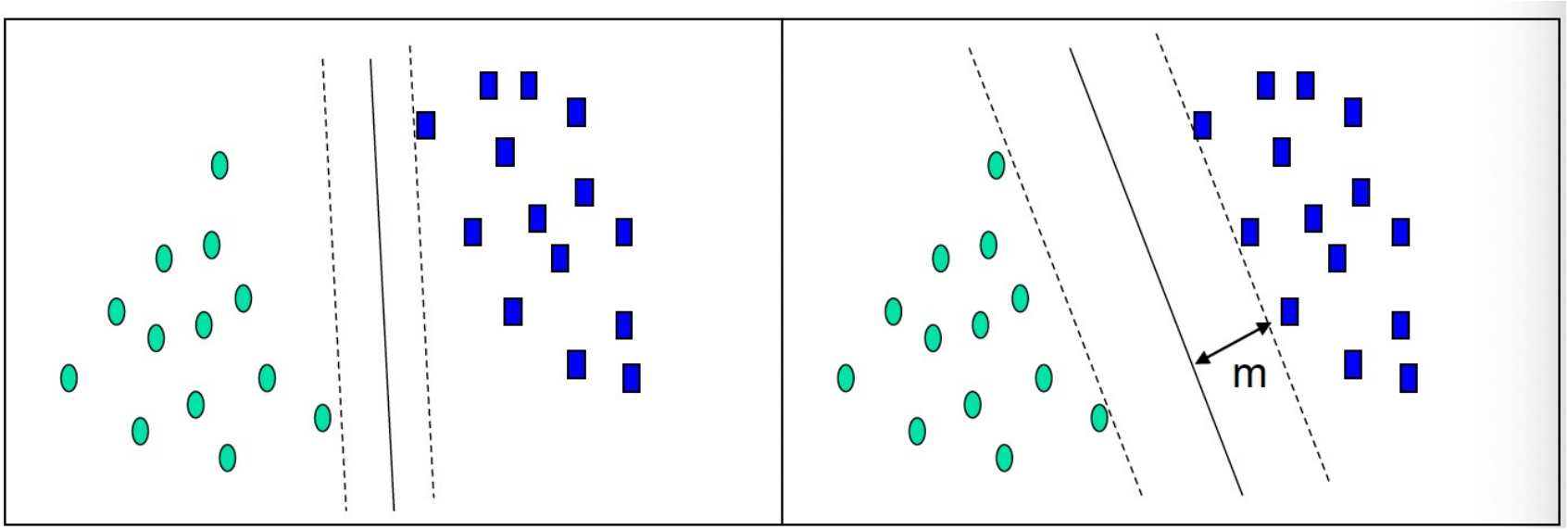


SVM - Ideas

- Goal: we want to find a decision boundary (e.g., solid line) that separates the green dots from the blue squares
- Support vectors: data points closest to the decision boundary that influence the position/orientation of the decision boundary

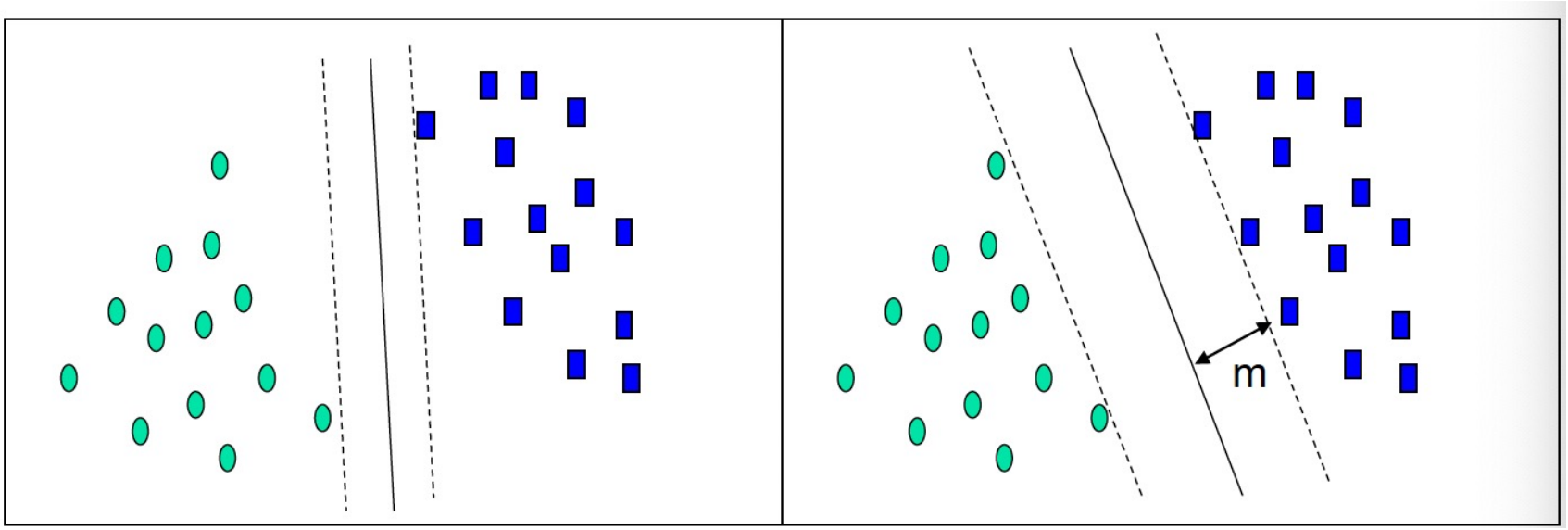


SVM Hyperplanes



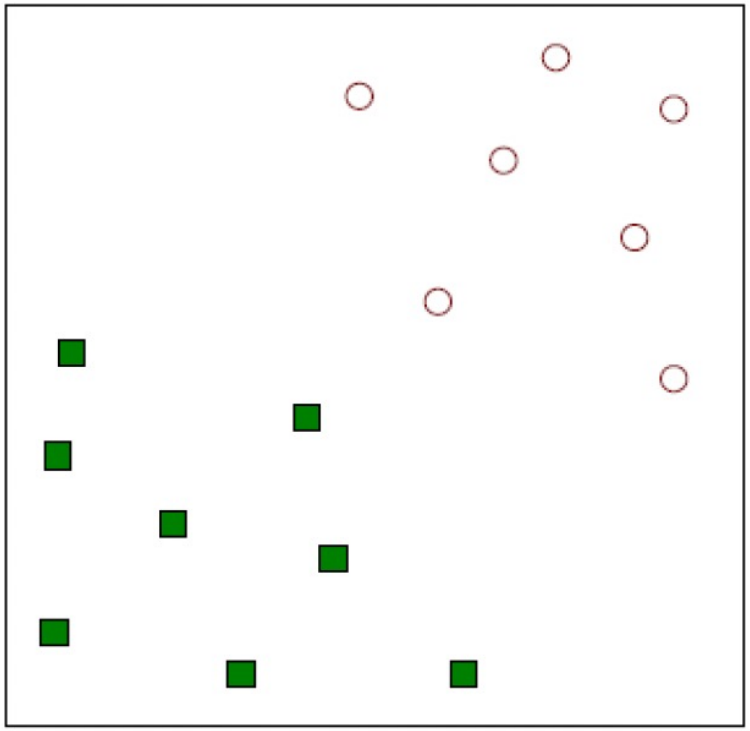
- Consider training tuples $\{(X_i, y_i)\}_{i \in \mathcal{D}}$ where X_i is the feature and y_i is the associated label
- Infinite decision boundaries (hyperplanes) separating the two classes
- We want to find the best hyperplane $\rightarrow$ how to define best?
  - SVM searches for the hyperplane with the largest margin

SVM Hyperplanes



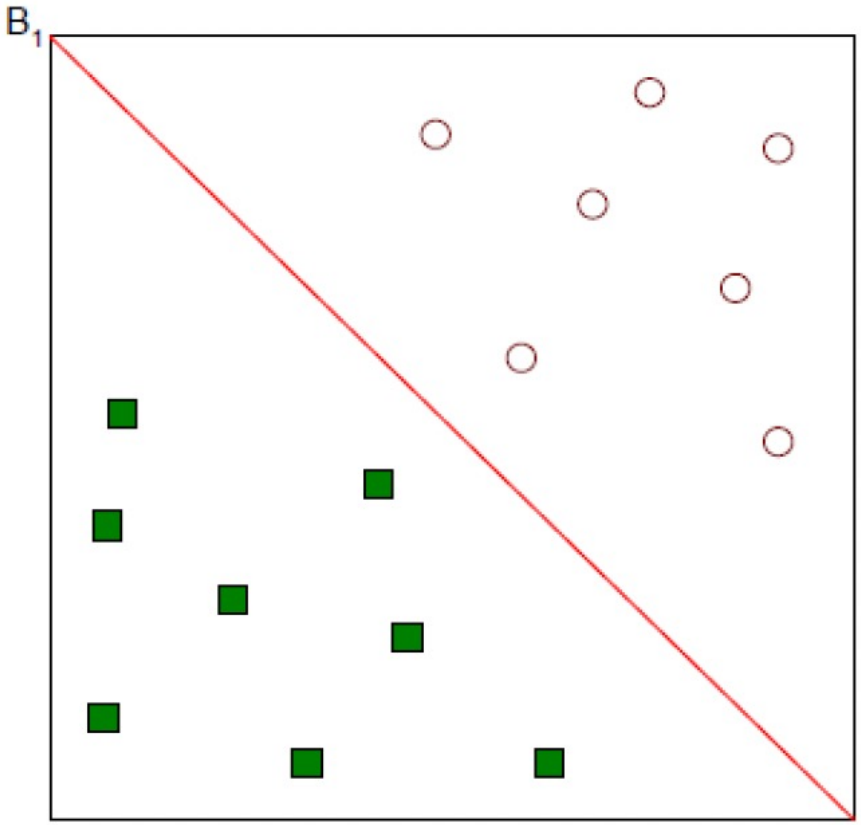
- Consider training tuples $\{(X_i, y_i)\}_{i \in \mathcal{D}}$ where X_i is the feature and y_i is the associated label
- Infinite decision boundaries (hyperplanes) separating the two classes
- We want to find the best hyperplane $\rightarrow$ how to define best?
  - SVM searches for the hyperplane with the largest margin

SVM: Linearly Separable



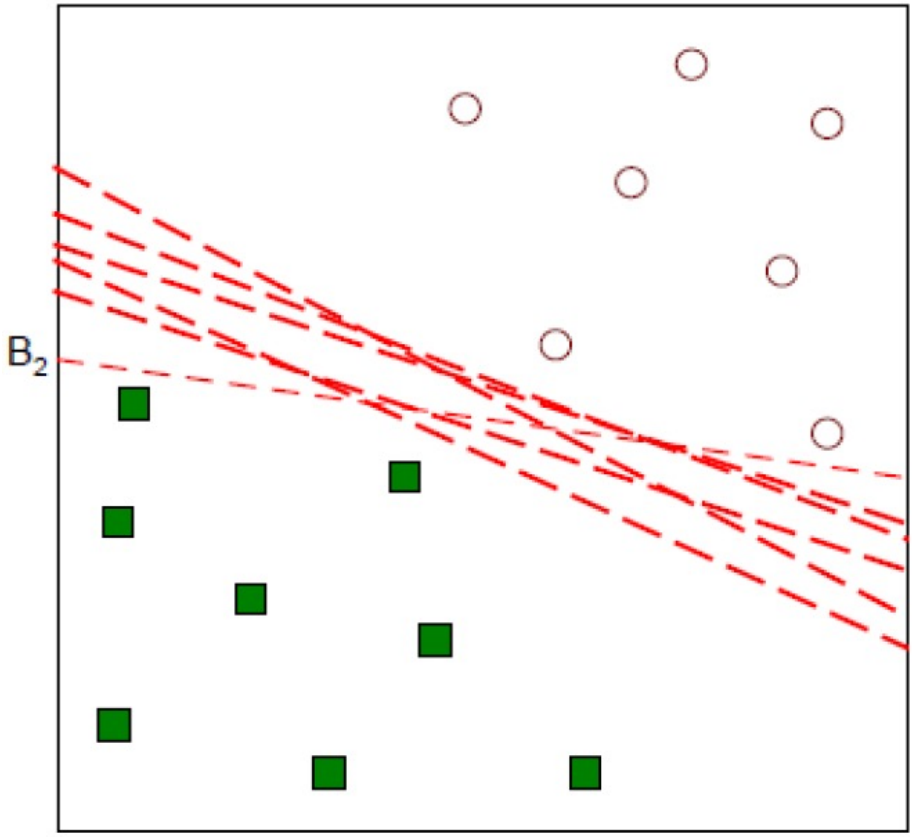
- Find a linear hyperplane that separates the two classes

Linear Separating Hyperplane



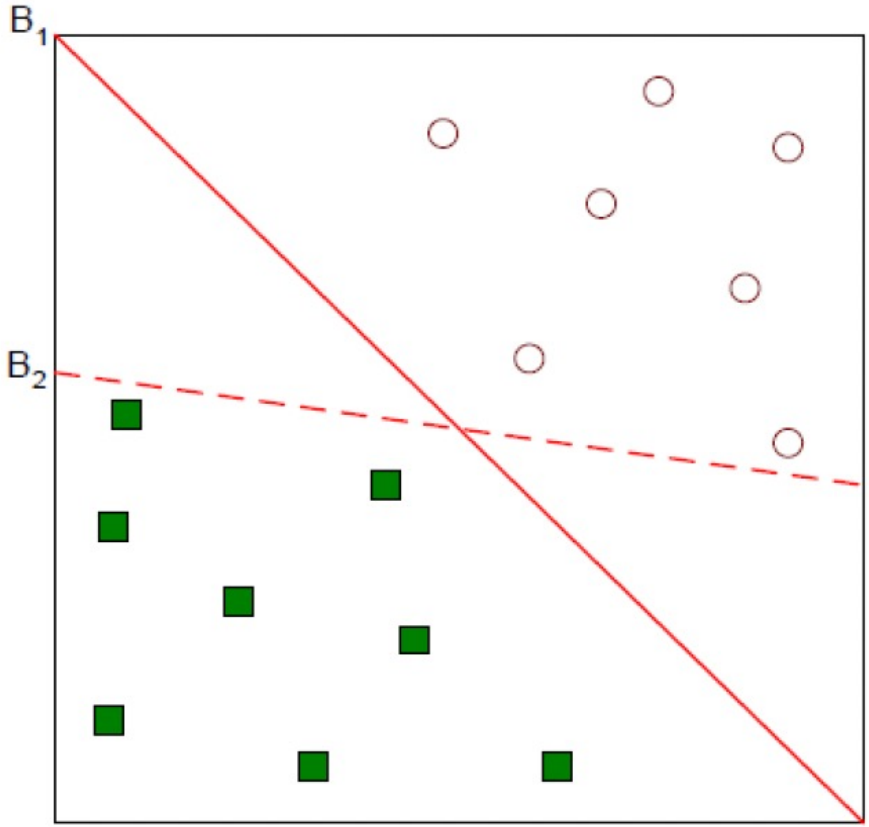
- One candidate solution

Other Linear Separating Hyperplanes



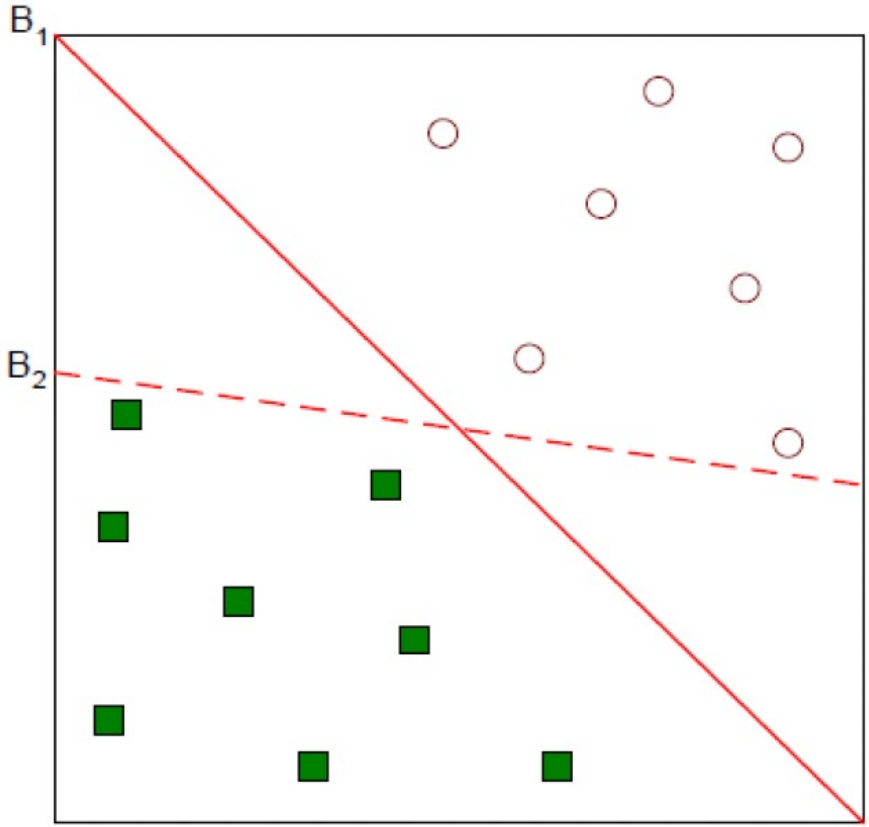
- Other candidate solutions

Finding a Good Hyperplane



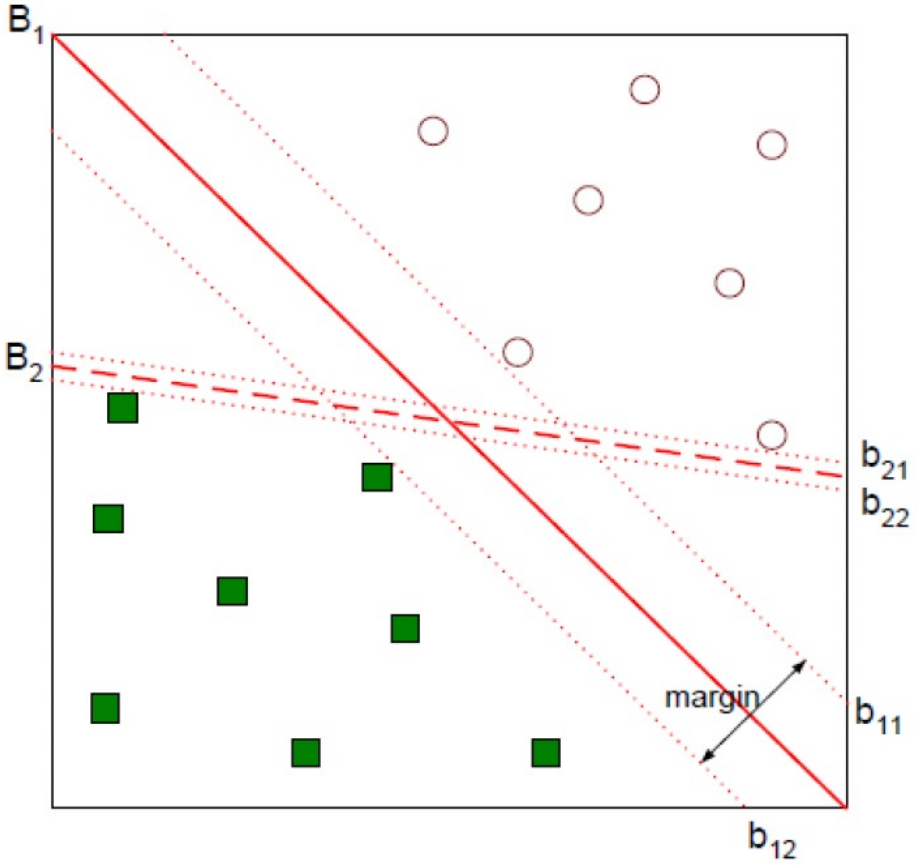
- Which one is better? B1 or B2?
- How do you define better?
  - How does SVM define better?

Finding a Good Hyperplane



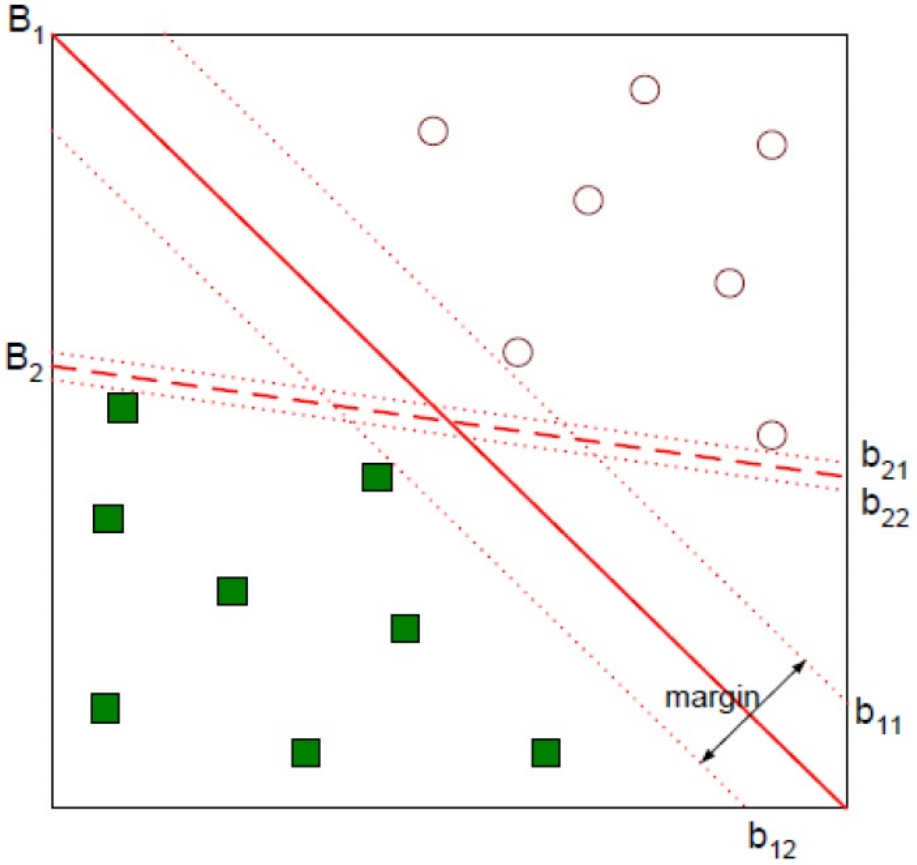
- Which one is better? B1 or B2?
- How do you define better?
  - How does SVM define better?

SVM Finds Maximum Marginal Hyperplane



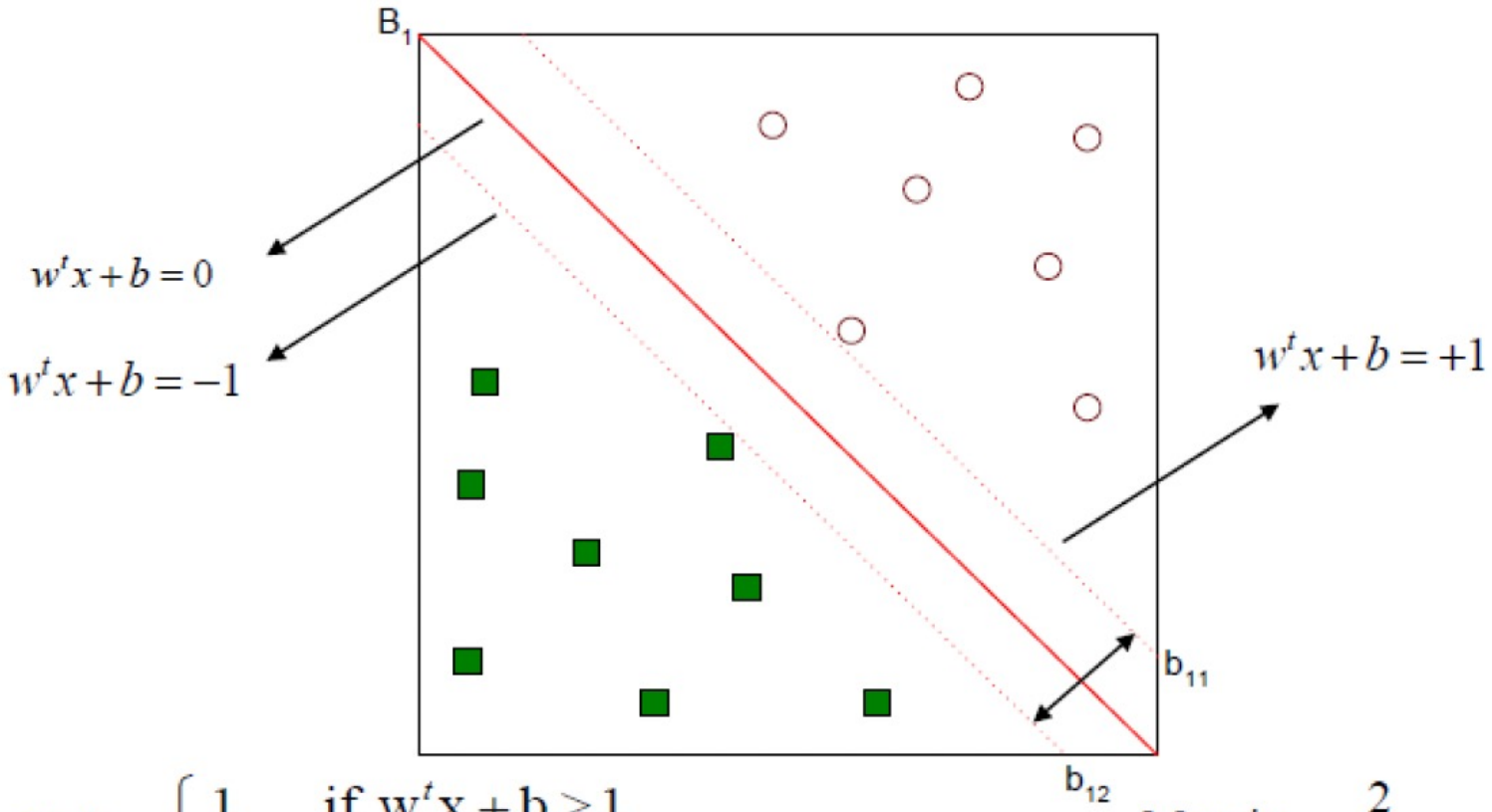
- SVM finds hyperplanes that maximizes the margin
  - B1 is better than B2

SVM Finds Maximum Marginal Hyperplane



- SVM finds hyperplanes that maximizes the margin
  - B1 is better than B2

Calculating the Margin



$$f(x) = \begin{cases} 1 & \text{if } w^t x + b \geq 1 \\ -1 & \text{if } w^t x + b \leq -1 \end{cases}$$

$$\text{Margin} = \frac{2}{\| w \|}$$

Forming An Optimization Problem

- We want to maximize the margin, i.e., $\frac{2}{||w||^2}$
- Subject to the following constraint

$$f(x_i) = \begin{cases} 1, & \text{if} \quad w^t x_i + b \geq 1, \\ -1, & \text{if} \quad w^t x_i + b \leq -1 \end{cases}$$

- This is a constrained optimization problem
  - Analytical solution: $w = \sum_{i=1}^{N_{sv}} \alpha_1 y_i x_i$
    - N_{sv}: # support vectors, α_i coefficients
    - Derivation omitted. Interested readers please refer to Boyd S, Vandenberghe L. Convex optimization. Cambridge university press, 2004. Sec. 4.4
  - Numerical solution: python package CVXPY for convex optimization
    - Documentation: https://www.cvxpy.org/

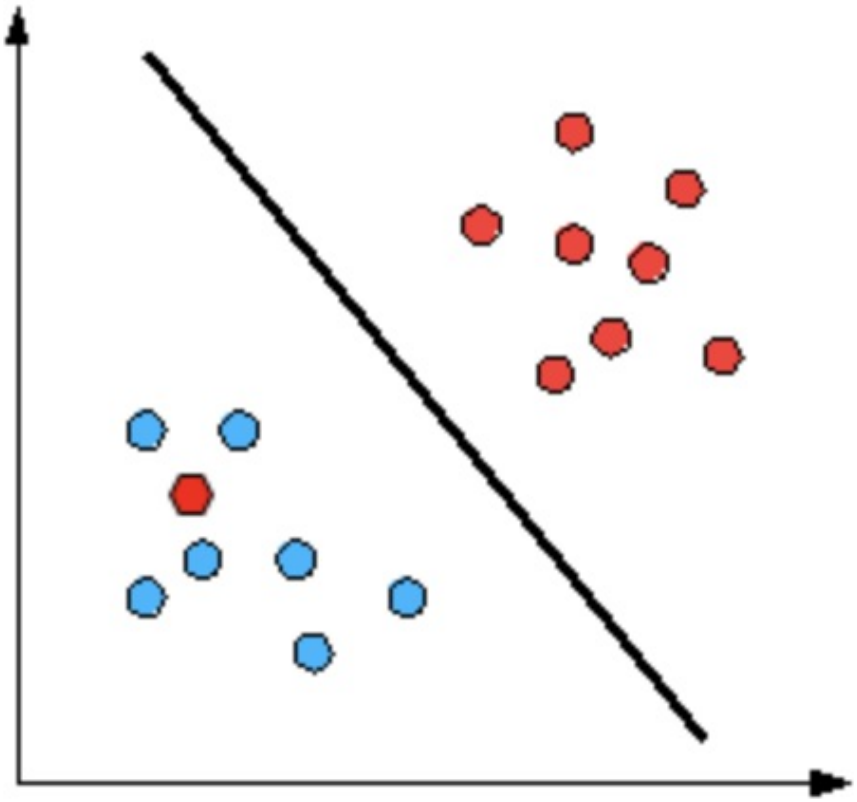
Forming An Optimization Problem

- We want to maximize the margin, i.e., $\frac{2}{||w||^2}$
- Subject to the following constraint

$$f(x_i) = \begin{cases} 1, & \text{if} \quad w^t x_i + b \geq 1, \\ -1, & \text{if} \quad w^t x_i + b \leq -1 \end{cases}$$

- This is a constrained optimization problem
  - Analytical solution: $w = \sum_{i=1}^{N_{sv}} \alpha_1 y_i x_i$
    - ★ N_{sv}: # support vectors, α_i coefficients
    - ★ Derivation omitted. Interested readers please refer to Boyd S, Vandenberghe L. Convex optimization. Cambridge university press, 2004. Sec. 4.4
  - Numerical solution: python package CVXPY for convex optimization
    - ★ Documentation: https://www.cvxpy.org/

SVM - Not Linearly Separable



- What if classes are not linearly separable? → Kernel method
- SVM maps input features into a high-dimensional space, where data non-linearly separable becomes linearly separable
  - Calculate the dot product of data points in higher-dimensional space (to be detailed in later lecture on kernel methods)
  - SVM finds hyperplane that maximizes the margin

Discussion of SVM

- Pros
  - Effective in high dimensional space (e.g., many features)
  - Effective when # dimensions is larger than # samples
  - Memory efficient as it uses a subset of training samples (support vectors) in decision boundaries
    - If all other training examples are removed and the training is repeated, the same separating hyperplane would be found
  - Good flexibility with kernel methods
- Cons
  - Not scalable to big data
    - Needs to compute the (Gram) matrix for all pairs of data samples, hence computationally expensive
  - Hard to interpret, especially when using kernels

Discussion of SVM

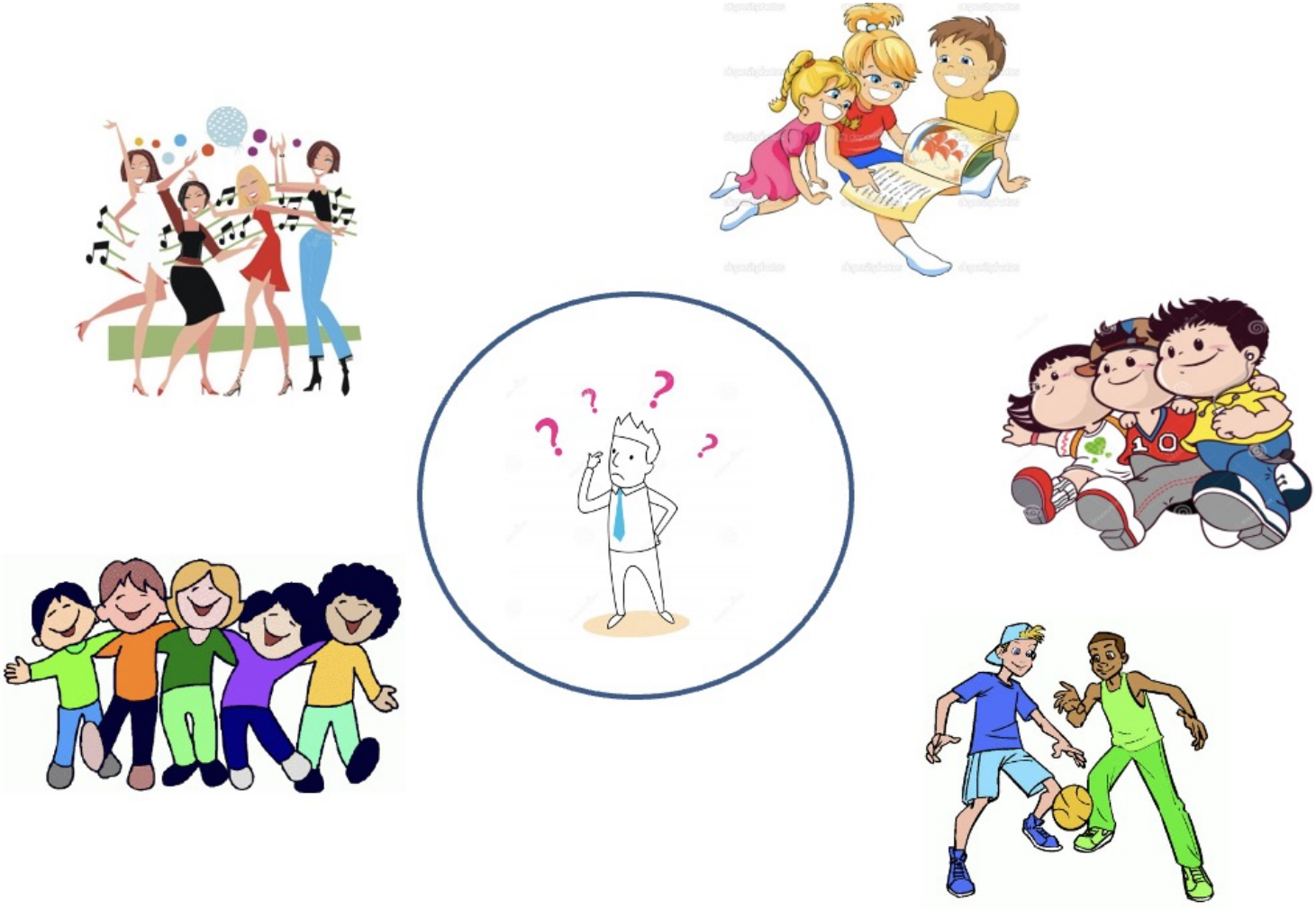
- Pros
  - Effective in high dimensional space (e.g., many features)
  - Effective when # dimensions is larger than # samples
  - Memory efficient as it uses a subset of training samples (support vectors) in decision boundaries
    - If all other training examples are removed and the training is repeated, the same separating hyperplane would be found
  - Good flexibility with kernel methods
- Cons
  - Not scalable to big data
    - Needs to compute the (Gram) matrix for all pairs of data samples, hence computationally expensive
  - Hard to interpret, especially when using kernels

k-Nearest Neighbors (KNN)

- A very useful classification algorithm for pattern recognition
- It stores all available samples and classifies new samples based on a similarity measure (e.g., distance)

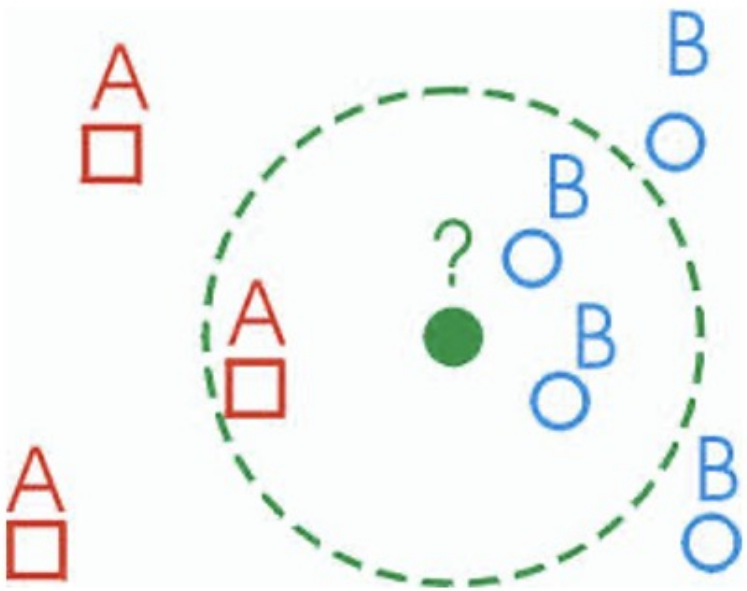
KNN: Simple Analogy

- Tell me about your friends (who your neighbors are) and I will tell you who you are

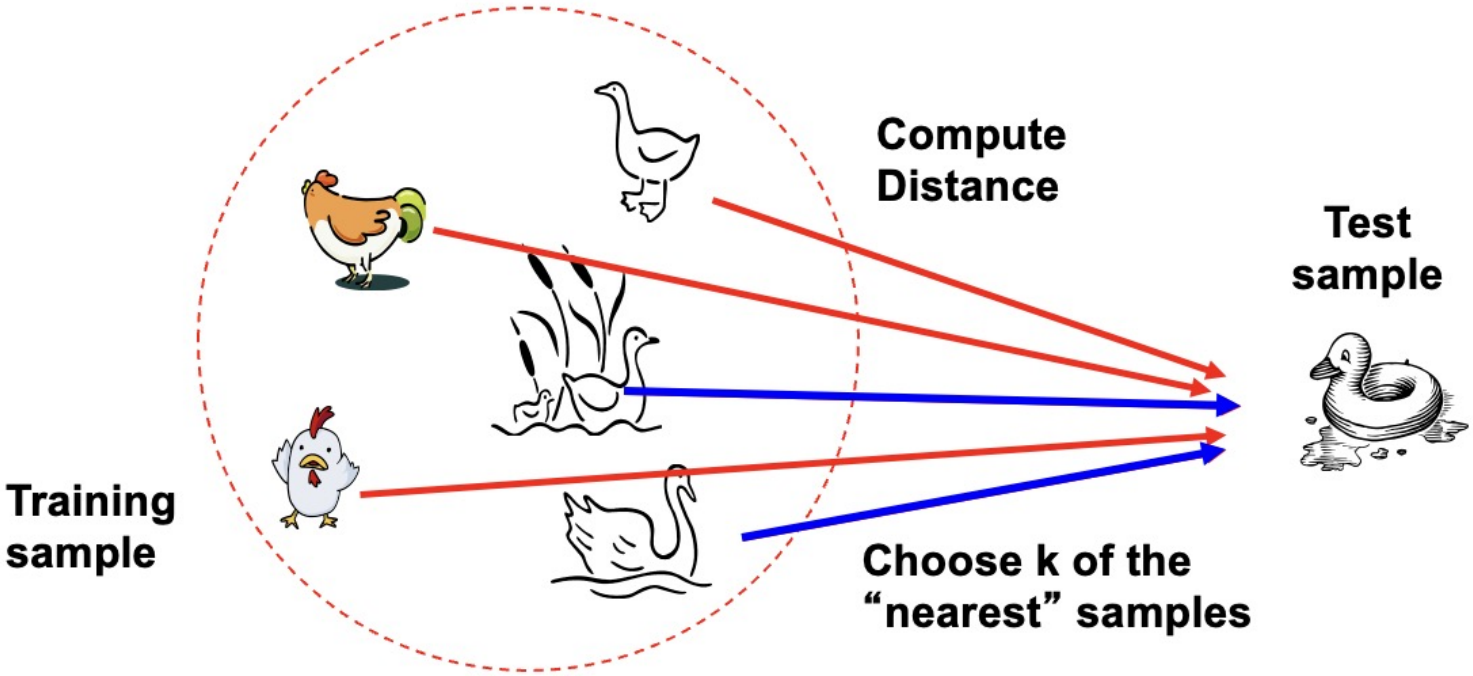


KNN: Classification Approach

- An object (a new sample) is classified by a majority vote for its neighbor classes
- The object is assigned to the most common class among its k nearest neighbors, measured by a distance function



Distance Measures



- Euclidean: $\sqrt{\sum_{i=1}^{k}(x_i - y_i)^2}$
  - Commonly used
- Manhattan: $\sum_{i=1}^{k} |x_i - y_i|$
- Minkowski: $(\sum_{i=1} |x_i - y_i|^q)^{1/q}$

KNN Algorithm

1. Choose the number of neighbors k
2. Calculate distance
   - For a given input data point, calculate the distance between this point and all other data points in the training set
3. Identify the nearest neighbors
4. Vote on the class
   - assign the new sample to the class that is most common among the k nearest neighbors (majority vote)
5. Repeat for all input data samples

Example

Customer	Age	Income	No. credit cards	Class
George	35	35K	3	No
Rachel	22	50K	2	Yes
Steve	63	200K	1	No
Tom	59	170K	1	No
Anne	25	40K	4	Yes
John	37	50K	2	?

- Goal: classify John's class assuming $k = 3$

Example - Cont'

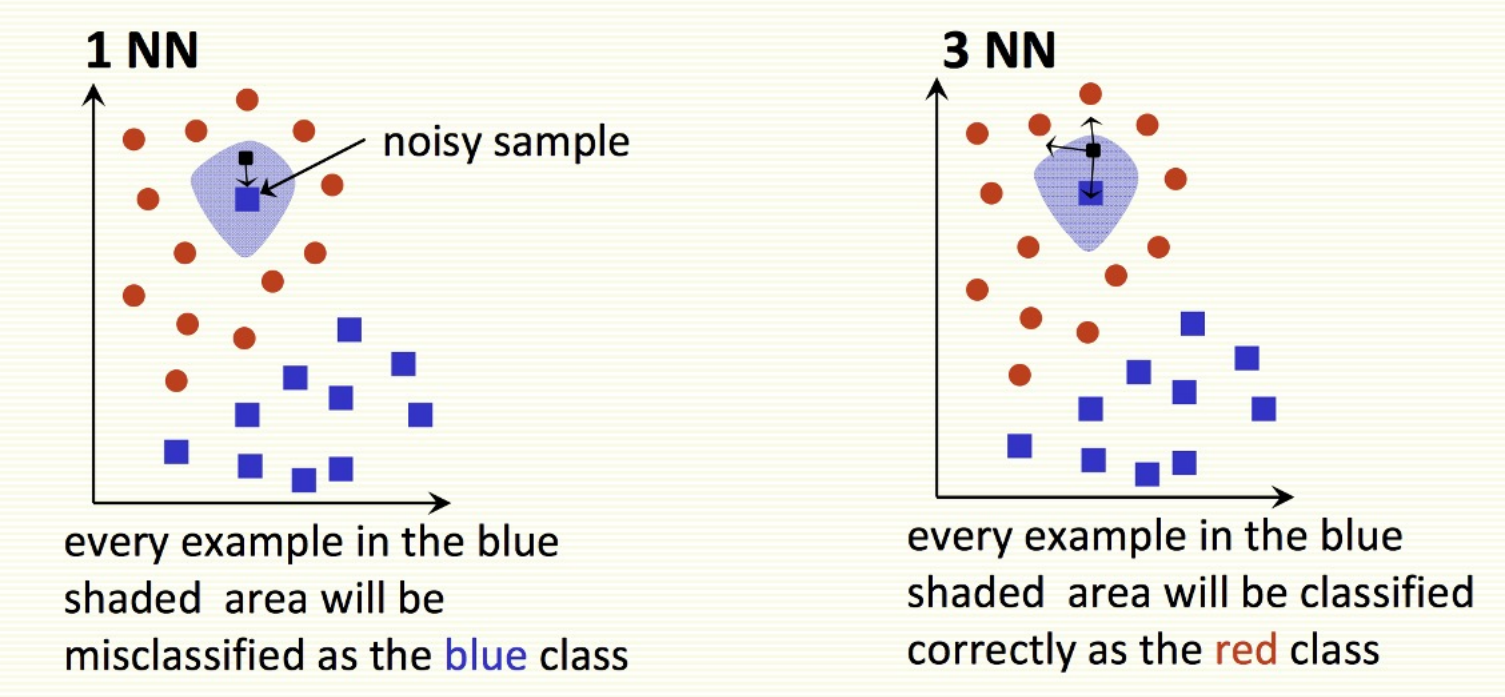
Customer	Age	Income	No. credit cards	Class
George	35	35K	3	No
Rachel	22	50K	2	Yes
Steve	63	200K	1	No
Tom	59	170K	1	No
Anne	25	40K	4	Yes
John	37	50K	2	YES

Distance from John
sqrt $[(35-37)^2+(35-50)^2+(3-2)^2]=15.16$
sqrt $[(22-37)^2+(50-50)^2+(2-2)^2]=15$
sqrt $[(63-37)^2+(200-50)^2+(1-2)^2]=152.23$
sqrt $[(59-37)^2+(170-50)^2+(1-2)^2]=122$
sqrt $[(25-37)^2+(40-50)^2+(4-2)^2]=15.74$

- Calculate John's distances to five persons
- 3 nearest neighbors: George (No), Rachel (Yes), and Anne (Yes)
- A majority vote leads to Yes

Hot to Choose k?

- If k is too small, KNN is sensitive to noise
- Larger k works well. But if k is too large, KNN may include majority points from other classes



- Suggestion: $k < \sqrt{n}$, where n is # samples

KNN Feature Normalization

- Distance between neighbors could be dominated by some features with relatively large numbers
  - e.g., income of customers in the "John" example
- Arises when two features are in different scales
- Important to normalize these features
  - Mapping values to numbers between 0 and 1

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i}$$

Discussion of KNN

- Pros
  - Simple and intuitive
  - Can be applied to data of any distribution
  - Good classification if # samples is large
- Cons
  - Takes more time to classify a new sample
    - ★ Need to calculate and compare distance from new sample to all other samples
  - Choice of k can be tricky
  - Need large number of samples for accuracy

Summary

- Support Vector Machines
  - classifies data by finding the optimal hyperplane that maximizes the margin between different classes in a (transformed) feature space
- k Nearest Neighbor
  - classifies a data sample by voting on the outcomes of its nearest k neighbors in the feature space

After-Class Reading

- M¨uller A C, Guido S. Introduction to machine learning with Python: a guide for data scientists. " O'Reilly Media, Inc.", 2016.
  - Sec. 2.3.2
  - Sec. 2.3.7
- Boyd S, Vandenberghe L. Convex optimization. Cambridge university press, 2004. (optional, will not be tested)
  - Sec. 4.4

Implementation on Jupyter Notebook

- Support Vector Machines
- KNN